

Student Achievement Showcase & Reporting Portal

A verified college achievement wall, backed by a reliable faculty record system.

Product requirements • Technology decisions • UX specification • Delivery plan

Prepared 6 October 2026 | Research checked 6 October 2026

Status: proposed implementation baseline, ready for stakeholder review. The working title is descriptive; final branding is still open.

Recommended build: Next.js + TypeScript + Tailwind CSS + shadcn/ui, with React Hook Form and Zod for submissions, and Supabase for PostgreSQL, authentication, and private file storage.

Core journey

Student records an achievement → faculty checks the evidence → approved, consented content appears publicly → faculty exports a consolidated report.

Based on the supplied handwritten concept. Faculty approval, privacy controls, draft recovery, audit history, and explicit reporting rules are proposed additions—not features assumed to exist already. All limits, targets, and schedules below are planning assumptions unless identified as documentation facts.

Product definition & scope

Problem and intended outcome

Achievement evidence is scattered across messages, email, and individual devices. Students lack a consistent place to showcase their work; faculty repeatedly chase certificates and reconstruct reports. The portal creates one structured record per achievement, with a verified public presentation and a separate internal evidence trail.

Audience	Job to be done
Student	Submit participation or a win, recover unfinished work, track review status, and receive recognition.
Faculty reviewer	Validate evidence, request corrections, publish eligible achievements, and retrieve records.
Department administrator	Manage reviewer access, categories and reporting periods; resolve disputes and oversee data quality.
Public visitor	Browse approved achievements across Technical, Non-Technical, Arts, and Sports.

Release boundary

Priority	Included requirements
P0 • launch	Verified student accounts; protected faculty access; submission form; private evidence uploads; review workflow; public category showcase; search and filters; faculty CSV/PDF reports; audit trail; responsive and accessible UI.
P1 • after pilot	Authenticated reactions; richer team co-ownership; saved report presets; email digests; optional XLSX export; featured achievements.
P2 • later	Legacy bulk import; institutional SSO; student portfolio links; optional OCR to prefill fields with human confirmation.
Out of scope	Event registration, attendance, marks, fee payments, ERP replacement, chat, public comments, AI-based certificate authenticity claims, and multi-college SaaS.

Assumed pilot and measurable outcomes

Start with one department, English UI, up to 1,000 accounts and 5,000 achievement records. Test against 100 concurrent browsing sessions. These are sizing assumptions, not proven capacity claims.

Pilot targets: $\geq 80\%$ of started submissions reach review within seven days; median active form completion ≤ 5 minutes excluding uploads; $\geq 90\%$ of submissions reviewed within three working days; a 1,000-row report ready within 30 seconds; zero unauthorized evidence disclosures. Establish the baseline during a two-week pilot.

Technology decision

“Best” means the strongest fit for a small React-capable team building forms, moderation, a public gallery, and relational reports. This recommendation favors one TypeScript application and managed infrastructure. It is an engineering judgment informed by the official documentation—not a performance benchmark.

Layer	Selection and rationale
Web application	Next.js App Router + React + TypeScript. Render public pages on the server; use client components for forms, filters, and reactions. Server/client boundaries are documented in [1].
UI and styling	Tailwind CSS + shadcn/ui. Own and adapt component code; use one consistent primitive family, Base UI for the proposed build. Add only the components required [2].
Forms	React Hook Form + Zod + shadcn Field components. Conditional fields, errors and reusable schemas; repeat validation at the server boundary [3].
Data and identity	Supabase PostgreSQL, Auth and Storage. SQL suits student-achievement-review relationships. RLS and storage policies provide defense in depth [6–7, 16].
Backend logic	Next.js route handlers calling a server-only service layer; SQL migrations and transactional database functions for submission, moderation and audit writes. No separate Express service in the default build.
Lists and reports	TanStack Table with shadcn Table for faculty sorting/pagination [11]. Server-generated CSV and a controlled PDF template; evaluate a maintained PDF library in the first implementation spike.
Motion and testing	Native CSS scrolling and transitions first; Motion only for optional richer transitions [8–9]. Playwright for browser flows [15]; unit tests for state transitions and validation.
Hosting and operations	Managed Next.js hosting plus a college-approved Supabase project; separate staging and production. Pick region, plan, email provider and background-job runtime after institutional review.

Version policy

Use supported stable releases compatible with each other at project kickoff; pin exact versions in the lockfile. Confirm the selected shadcn primitive family and form resolver compatibility in a small spike. Review dependency security updates weekly and before each release. Do not combine examples from incompatible framework majors.

Alternatives & UI library research

Option	Trade-off	Decision
Next.js + Supabase	One main language; managed identity and storage; SQL reporting. Requires careful RLS and cache design.	Recommended for the standalone product.
React/Next.js + Spring Boot + PostgreSQL	Explicit Java backend, Spring Security and service layer; extra API deployment and validation contracts [12].	Choose if Java assessment or college IT requires it.
Fully custom backend + self-managed database/storage	Maximum infrastructure control, but more backup, patching and identity work.	Only if the institution can operate it.
Form tool + spreadsheet	Fast intake prototype, but weak control of moderation, evidence access and public card presentation.	Useful for validating fields, not the full product.

Component library choice

Library	Fit for this project
shadcn/ui [2–5]	Selected: customizable form, dialog, badge, table, skeleton and spinner components. The team must maintain copied/customized code and verify accessibility after changes.
Material UI [13]	Strong alternative for a conventional admin-heavy interface. Its Material Design system may need more visual adaptation for the public showcase. Avoid mixing it with shadcn in the same product.
Custom components only	Unnecessary effort for focus management, dialogs, errors and keyboard behavior. Reserve custom work for achievement cards and the publication preview.

Smooth-scrolling decision

Use normal browser scrolling, with CSS smooth behavior for anchor navigation and deliberate “back to top” actions. CSS scroll-behavior governs navigation/programmatic scrolling, not a replacement for ordinary wheel behavior [8].

Lenis is an available JavaScript smooth-scroll option [10], but is not needed for the MVP. If a future showcase design requires it, test anchors, keyboard scrolling, touch, nested panels and reduced-motion behavior before adoption. Forms and faculty tables should retain native scrolling.

If this is a Java-course project: keep the same PRD and frontend. Move the service layer and APIs to Spring Boot, use Spring Security and Bean Validation, and keep PostgreSQL plus protected object storage. Choose one backend owner; do not duplicate business rules across Java and Next.js.

Roles, visibility & navigation

Capability	Visitor	Student	Faculty	Admin
Read public showcase	Yes	Yes	Yes	Yes
Create/edit own draft	No	Yes	If also student	If also student
View private evidence	No	Own only	Assigned scope	Authorized scope
Approve/request changes	No	No	Assigned scope	Authorized scope
Publish/unpublish	No	No	Assigned scope	Authorized scope
Export full records	No	Own record later	Assigned scope	Authorized scope
Assign roles/settings	No	No	No	Yes
React to public cards (P1)	No	Yes	Yes	Yes

Roles come from protected memberships controlled by administrators. A selected “faculty” option or a college email address must never grant reviewer rights. Students may hold multiple roles, but may not approve their own submissions.

Proposed routes

Surface	Routes and purpose
Public	/ — showcase; /achievements — full searchable list; /achievements/[slug] — public detail; /about — purpose and submission guidance.
Student	/sign-in; /dashboard; /submissions/new; /submissions/[id]/edit; /submissions/[id] — private detail, evidence and review history.
Faculty	/faculty/review; /faculty/review/[id]; /faculty/records; /faculty/reports — queue, evidence inspection, moderation and exports.
Administrator	/admin/members; /admin/settings; /admin/audit — assignments, controlled values and audit search.

Visibility boundary

Public cards contain display name, department/year if permitted, event, category, outcome, date, approved description and approved photos. Registration numbers, email addresses, certificates, internal comments and export files remain private. This intentionally changes the sketch’s public registration-number idea to a safer default.

Faculty permissions are department-scoped. Team records involving other departments remain with the submitting department in MVP; reviewers must not gain access to other students’ unrelated records. Multi-department moderation is a later requirement.

Submission & moderation lifecycle

State	Allowed action and next state
Draft	Owner edits/deletes; successful validated submission → Submitted. Drafts are not counted in official achievement totals.
Submitted	Assigned faculty opens review → Under review. Owner may withdraw → Withdrawn before a decision.
Under review	Reviewer requests changes → Changes requested; rejects with reason → Rejected; or verifies → Approved.
Changes requested	Owner edits and resubmits → Submitted, creating a new immutable revision. Faculty sees the prior feedback.
Approved	Verified internal record. Eligible public projection + explicit publication consent → Published; otherwise remains internal.
Published	Public projection visible. Faculty may unpublish with reason → Approved; owner may request correction or withdraw publication consent.
Rejected / Withdrawn	Private retained history. Student can create a corrected draft; resubmission must follow the same evidence checks.

Rules that prevent inconsistent records

- Every state change records actor, timestamp, prior state, new state, revision and reason where applicable. Notification creation and state changes commit together.
- Approval verifies a specific revision. Published content is an immutable approved snapshot. Subsequent student edits create a new draft revision; the last approved snapshot stays visible until replacement, unless it is disputed or consent is withdrawn.
- Approval and publication may be one faculty action only when all publication conditions are met. Otherwise show “Approve for internal record” and explain why publication is unavailable.
- Use optimistic concurrency: if another reviewer changed the revision/state, reject the stale action with a conflict message and reload. A review assignment reduces duplication but never replaces database checks.
- Unpublishing removes the public projection and invalidates public caches. “Remove” means audited unpublishing or soft deletion; permanent purge follows the college’s retention policy.

Definition of verified

Faculty confirms that the evidence reasonably supports the student, event, date and outcome. A “Faculty verified” badge communicates this review; it must not imply cryptographic or issuer-level authenticity. OCR, if added, only helps data entry.

Website form: fields & validation

FR-01: A mobile-first, three-step form: 1) student and event, 2) result and evidence, 3) preview and consent. Show step labels, Save draft, Back and Next. Final CTA: “Submit for verification”.

Field	Rule / proposed limit
Student name / registration no.	Required; prefilled from verified profile and read-only. Corrections go through profile verification. Registration no. unique within college; exact pattern configured by institution.
Department / batch / year	Required controlled values. Record an event-time snapshot so later promotions do not rewrite historical reports.
Event name / organizer	Required, 3–150 characters each; trim whitespace. Avoid arbitrary capitalization changes.
Event date	Required past or current date; reject future dates. Use event date for reporting, store as a date independent of timestamp timezone.
Category / level	Category: Technical, Non-Technical, Arts, Sports. Level: intra-college, inter-college, state, national, international; faculty may correct it.
Outcome / rank	Participation or Award/Win required. For Award/Win, require rank (positive integer) or award title (2–100 characters). A special award need not have a numeric rank.
Participation type	Individual or Team. Team name optional; member names and registration numbers captured privately; deduplicate repeated members. Proposed cap: 20 members.
Description	Required plain text, 30–1,500 characters. Public preview uses approved text; escape output. No arbitrary HTML embeds.
Certificate / supporting proof	At least one evidence file required to submit. If certificate is unavailable, allow official result proof or event photo plus explanation; faculty decides sufficiency.
Showcase photos / alt text	Optional, up to 5 photos; each requires descriptive alt text before publication. A designed category placeholder appears if no photo is approved.
Consent / declaration	Accuracy declaration required to submit. Public showcase consent is separate and optional, unchecked by default. Record consent version and timestamp.

Validation appears beside fields and in a linked error summary. Validate on blur and submit; avoid warning while a user is still typing. The server repeats every rule. Unicode names are accepted; college identifiers are normalized using institution-approved rules.

Form resilience & file handling

FR-02 • Drafts and submission reliability

Create a server draft after the first valid input. Autosave after approximately two seconds of inactivity, with “Saving...”, “Saved at ...” and “Could not save — Retry” states. The form remains editable during a save. Use a version token to prevent two tabs overwriting each other silently.

On session expiry, preserve in-memory text and offer reauthentication. Never promise that unsaved content survived a reload. Display a leave-page warning when unsaved edits exist. Uploaded files are linked by server IDs; do not persist certificate bytes in localStorage.

Disable repeated final submission while pending and use an idempotency key server-side. Retries with the same key return the original result. Show a submission reference and status after success. A submission cannot enter review while any required upload is incomplete or quarantined.

FR-03 • Upload contract

Setting	Proposed launch behavior
Evidence	PDF, JPEG or PNG; maximum 3 files at 10 MB each.
Showcase photos	JPEG, PNG or WebP; maximum 5 at 5 MB each. Explain unsupported HEIC conversion if encountered.
Total limit	Maximum 50 MB per submission, enforced before upload and again server-side; institution-level quotas configurable.
Experience	Click-to-select plus drag-and-drop; thumbnail, filename, size, real byte progress, cancel, remove and retry per file. Never show a fabricated percentage.
Processing	Private quarantine → signature/type and size checks → malware scan → clean private object. Re-encode public image derivatives and remove location metadata.
Failure behavior	Invalid file: specific error without losing the form. Scanner unavailable: remain processing; no faculty download or publication until cleared. Abandoned uploads cleaned after 24 hours.

Storage and evidence privacy

Use short-lived signed upload/download authorizations scoped to one object and authorized user. Store generated object keys rather than user filenames as paths. Certificates remain private; only sanitized, reviewer-approved photo derivatives may be served on the public surface. Follow an allowlist, signature validation, size restrictions and scanning guidance [14].

For published images, use an authorization-aware media route that checks publication status and returns private origin bytes without long-lived public caching. On unpublish, revoke that route immediately. Previously downloaded images cannot be recalled. Database backups and file backups must be handled separately [17].

Public showcase requirements

FR-04 • Homepage and browsing

A compact header contains the college identity, Achievements, About, and Sign in / Dashboard. A concise introduction leads to “Explore achievements” and “Submit an achievement”. Avoid a full-screen hero that pushes student work below the fold.

Preserve the sketch’s four category shelves. Each shows up to six recent published records with an accessible horizontal shelf and “View all”. Desktop arrow buttons and touch swiping are supported; no automatic carousel rotation. The full listing uses a responsive grid and explicit pagination.

Element	Required behavior
Achievement card	4:3 image or category placeholder; student/team display name; event; date; category; participation/award label; faculty-verified badge. Clicking opens a dedicated detail URL.
Filters	Category, outcome, event academic year and department. Search event, approved description and public display names. Query parameters preserve filters when sharing a link or returning from detail.
Sorting	Default: event date descending, then stable ID. Optional: recently published. Popularity is not the default and never affects official achievement metrics.
Pagination	12 records per page, server-side filtering. Keep previous results visible during updates and announce the updated result count.
Detail page	Full approved description, event and result context, accessible photo gallery, publication date and a correction/report link. No certificate links or private identifiers in HTML or metadata.
Empty / removed	No matches: Clear filters. No published data: neutral invitation to submit. Unpublished/deleted detail: generic unavailable response with a route back to browse.

FR-05 • Reactions, P1

Signed-in, verified users may choose Like, Love or Fire. One active reaction per user per achievement; switching replaces it and clicking the same reaction removes it. Enforce uniqueness in the database and rate-limit mutations. Anonymous visitors can see totals but must sign in to react. Optimistic UI rolls back on failure.

Acceptance criteria

Only the approved, consented revision is publicly readable. Direct URL access cannot reveal a draft. Filters yield consistent server totals. A keyboard user can reach every card and shelf control. Public APIs, image routes, page source, search snippets and social previews contain no registration numbers or evidence URLs.

Faculty workspace & reporting

FR-06 • Review queue

Show Submitted, Under review and Changes requested tabs; age, student, event, category and assigned reviewer; filters by department, year and outcome. Review detail places the submitted fields alongside a protected evidence viewer. Actions: claim review, request changes, reject, approve internally, approve and publish. Require reasons for rejection, correction requests and unpublication. No bulk approval in MVP.

FR-07 • Consolidated reports

Faculty filters by event-date range, academic year, department, batch, category, outcome, event level and verification/publication state. Default official report includes current Approved and Published records, excluding drafts, rejected and withdrawn records. Operational pending-review reports are clearly labeled separately.

Output	Contract
CSV • P0	One row per achievement with record ID, student registration/name, department/batch snapshot, event, organizer, date, category, level, outcome/rank, team size, reviewer and verification timestamp. Include no live certificate URLs.
PDF • P0	College heading, applied filters, generated-by/time, summary counts, detailed rows and page numbers. Use landscape pages for the record table. Public/publication status is distinct from verification status.
Team reporting	Main report counts one team achievement once. A separate member report produces one row per participant and labels the metric “participation records”. Never add these counts together.
Large exports	Up to 1,000 records: target ≤30 seconds. Above this, queued job with progress state, authorized download and retry. Cap MVP at 10,000 rows per job; request a narrower filter if exceeded.
Download controls	Recheck role/scope at job creation and download. Private export objects expire after 24 hours; signed links expire after 5 minutes. Log the export, filters and row count.

Report correctness

Generate totals and detail rows from the same database snapshot. Display event-date basis and institutional timezone (proposed Asia/Kolkata). Neutralize spreadsheet formula prefixes in text fields, escape CSV correctly, and prevent arbitrary HTML in PDF output. A report link cannot confer access to an unauthorized user.

Acceptance: UI count equals exported row count for identical filters; a team of five counts as one achievement and five participant rows; faculty cannot export another department; a failed job does not produce a misleading success notification.

Visual design & component system

Direction: a calm, contemporary college showcase: strong photography, clear typography, generous spacing and restrained achievement accents. The student experience should feel welcoming; faculty screens should prioritize scanning and completing work.

Token / pattern	Proposed specification
Color	Background #F8FAFC; surface #FFFFFF; primary #1D4ED8; text #0F172A; muted text #475569. Gold reserved for awards; verify all final foreground/background combinations.
Typography	One self-hosted sans-serif family or system stack; body 16 px, labels 14 px, H1 32–40 px desktop / 28 px mobile. No essential information in tiny captions.
Spacing and shape	4 px spacing scale; 16–24 px card padding; 12 px card radius; subtle borders and shadows. Maximum content width approximately 1,200 px.
Responsive layout	320–639 px: one card column; 640–1023 px: two; 1024+ px: three. Form stays one column on phones and at most two logical columns on desktop.
Actions	One primary CTA per section. Secondary outline buttons for Back/Save draft; danger styling only for destructive actions. Icons accompany labels rather than replace them.
Status	Use text + icon + color for Draft, Submitted, Under review, Changes requested, Approved, Published and Rejected.
Faculty table	Column visibility and server pagination; mobile summary rows open a detail page. Horizontal scrolling is acceptable for a genuine data table, not the entire screen.

Component inventory

shadcn: Button, Field, Input, Textarea, Select/Combobox, Checkbox, Calendar/Date Picker, Card, Badge, Tabs, Dialog/Alert Dialog, Sheet, Table, Pagination, Progress, Skeleton, Spinner, Tooltip and Toast.
Custom: AchievementCard, CategoryShelf, EvidenceUploader, ReviewPanel, StatusTimeline and ReportFilterBar. Use Lucide icons consistently.

Accessibility requirements

Target WCAG 2.2 AA [18]. Support keyboard-only completion, visible focus, labeled controls, linked error descriptions, logical headings and meaningful image alternatives. Dialogs trap focus and restore it on close. Aim for 44×44 px touch targets. Status and validation must not depend on color. Verify zoom/reflow and screen readers; a component library alone does not establish conformance.

Scrolling, loading & feedback UI

Situation	UI requirement
Initial public list load	Render page shell immediately and show 6–12 card-shaped skeletons with reserved image dimensions. Swap only the loaded region; avoid a blocking full-page spinner [5].
Button action	Inline 16–20 px Spinner plus “Submitting...” or “Approving...”. Keep button width stable and prevent duplicates. Announce result with a polite live region [4].
Upload	Determinate progress from transferred bytes; show “Processing evidence...” separately after transfer. A completed transfer does not mean verification or virus scanning is complete.
Search / filter change	Debounce typing about 300 ms. Keep existing results visible with a subtle updating indicator and cancel/ignore stale requests.
Long operation	After about 10 seconds, show explanatory text. After a defined timeout, offer status check or retry with the same idempotency key. Never spin indefinitely without guidance.
Report job	Queued → Generating → Ready / Failed. Show count-based progress only when measurable; allow users to leave and return to the report job.
Empty / error	Empty content is not loading. Provide a clear next action. Errors explain the failed action and retry route; preserve valid user input.

Motion rules

Normal wheel/touch scrolling stays native. Smooth anchor navigation respects sticky-header offset. Route changes place focus at the new page heading; browser Back restores listing position. Do not animate every card on every scroll.

Use 150–200 ms opacity/color transitions for feedback and at most 200–250 ms for panel entry. Motion is optional; CSS is sufficient for MVP. Respect prefers-reduced-motion: turn off smooth scrolling, nonessential transform movement and skeleton shimmer; replace spinner rotation with static progress text where appropriate [8–9].

Acceptance criteria

At 320 px width, keyboard focus remains visible and the page has no accidental horizontal overflow. Loading content does not shift card geometry. Screen readers announce action/result once, without repeated per-frame announcements. A slow or offline connection yields a recoverable state. No decorative loading delay is introduced to make animations visible.

Architecture & trust boundaries

Boundary	Responsibility
Browser	Renders UI and runs client validation. Holds no privileged server secret. Direct uploads require a scoped authorization from the backend.
Next.js public routes	Read only a safe published projection; cache public text briefly with explicit revalidation. Never fetch private records and hide fields only in the UI.
Authenticated routes	Verify identity and current membership, validate input, enforce ownership/scope and call server-only services. Private responses are not shared-cacheable [16].
PostgreSQL	Authoritative records, revisions, memberships, consent, review history and exports. Constraints and RLS restrict rows; transaction functions make transitions and audit writes atomic [6].
Object storage	Private evidence originals and sanitized photo derivatives. Storage policies and the media route prevent unauthorized reads [7].
Background worker	File scanning/derivatives, large exports and cleanup. Uses minimal scoped credentials, retry limits and idempotent jobs; alerts on stuck or failed work.

Read and write patterns

Use Server Components for initial public content; Client Components only where interactivity is needed [1]. Route all sensitive mutations through a consistent service boundary. Prefer user-scoped database requests with RLS; keep privileged credentials restricted to audited jobs or narrowly scoped administrative operations.

A `published_achievements` projection contains safe fields only. Publish/unpublish changes the projection in the same transaction as workflow state and audit. Public reads use that projection; originals and review comments are never available to anonymous roles. Publication withdrawal also invalidates detail/list caches and denies media access.

Authentication and deployment

MVP: verified college-email login with a roster/invitation match. Faculty accounts are administrator-provisioned; require MFA for reviewers/admins. Institutional SSO is an integration phase, not assumed available. Follow the current supported Supabase SSR session verification pattern [16].

Maintain development, staging and production environments with distinct databases, keys and buckets. Use synthetic test records outside production. Keep source/migrations in Git; CI runs lint, type checks, critical tests and build. Production data migration and deployment rollback must be documented. Choose a worker runtime that actually supports the scanner and PDF generator; do not assume an edge runtime can run all binaries.

Data model & integrity rules

Entity	Key attributes and constraints
profiles / memberships	Auth user ID; private registration no.; display name; institution/department; batch; role. Unique institution + registration no.; only admins change roles.
achievements	ID; owner; submitting department; workflow state; current revision; approved revision; public slug; version counter; soft-delete timestamp.
achievement_revisions	Achievement ID + revision no.; event, category, level, date, outcome/rank/title, description, event-time student/department snapshot. Immutable after submission.
participants	Revision ID; member profile/reference; private registration no.; display name; team role; public-name consent. Unique member per revision; nonconsenting names omitted publicly.
attachments	Revision ID; random object key; kind; MIME; bytes; checksum; scan status; alt text; approved derivative key. A clean scan is required before authorized viewing/publication.
reviews / audit_events	Actor, target revision, department scope, decision, reason, timestamp, old/new state and request ID. Append-only to normal application users.
consents	Subject; revision; scope (name/photo/showcase); policy version; granted/withdrawn time. Team member public identity requires that member's consent or an institution-approved basis.
published_achievements	Approved safe snapshot, public photo references and public member names only. One current public projection per achievement.
reactions / notifications	Unique user + achievement reaction; authenticated only. Notification recipient, event type, read time and deduplication key.
export_jobs / job_queue	Requester, permission scope, filter snapshot, status, result key, expiry, attempts and sanitized error. Jobs use idempotency keys.

Indexes and reporting semantics

Index achievements by department/state/created time and published records by category/event date/ID. Add a PostgreSQL search index over approved public text. Index participant and attachment foreign keys. Use server pagination and test query plans with the target dataset before adding a separate search engine.

A record represents an achievement by an individual or team, not the event itself. Two students independently participating in the same event are not automatically duplicates. Flag matching owner/team + event/date/category/outcome combinations for review; file checksum matches are warnings, not proof of fraud.

API contract & acceptance tests

Proposed REST-style contract for the service layer; the Java alternative can implement the same contract. All private endpoints recheck current role/scope. Use ISO date-only event fields and UTC timestamps, rendered in institutional timezone.

Endpoint / action	Authorization and result
GET /api/achievements	Anonymous safe projection; filters, sort, page and pageSize; bounded to 50 rows/page.
POST /api/submissions	Verified student; creates own draft; returns ID and version.
PATCH /api/submissions/:id	Owner + editable revision; expectedVersion required. Field errors returned structurally.
POST /api/uploads/authorize	Owner editable draft; allowlist/quota checks; single-object upload authorization.
POST /api/submissions/:id/submit	Owner; completed clean evidence; idempotency key; returns submission reference.
POST /api/reviews/:id/decision	Scoped reviewer; expectedVersion + revision + decision + reason; atomic transition and audit.
POST /api/achievements/:id/publication	Scoped reviewer to publish/unpublish; consent withdrawal by owner uses a separate protected action.
POST /api/reports + GET /api/reports/:id	Scoped faculty; filter snapshot → job; download authorized again at access time.
PUT /api/achievements/:id/reaction	P1; verified user; validated reaction enum or null; unique user/achievement.

Errors and security behavior

Use 400/422 for invalid input, 401 for missing login, 403 or non-enumerating 404 for unauthorized records, 409 for stale state/version, 413 for oversize files and 429 for rate limits. Return a safe error code, readable message and correlation ID; never return stack traces or storage secrets.

Release-critical scenarios

- Student A attempts to read/edit Student B's draft or file: denied through both API and storage routes.
- Two faculty members decide on the same revision: one succeeds; the stale decision gets a conflict.
- Double submit, refresh and network retry: one submission and one audit transition.
- Consent withdrawn from a published record: public page, list, metadata and media route no longer expose it within the defined removal window.
- A 1,000-record export reconciles to the filtered snapshot, and CSV text cannot execute formulas.

Quality, security & operations

Area	Proposed acceptance target
Performance	Public mobile p75: LCP ≤ 2.5 s, INP ≤ 200 ms, CLS ≤ 0.1 , aligned with Core Web Vitals thresholds [19]. Measure real traffic when available; use throttled lab testing before launch.
API / scale	At pilot load of 100 concurrent browsing sessions and 5,000 records: paginated read p95 ≤ 750 ms excluding client network; non-upload mutation p95 ≤ 1.5 s. Validate with load tests, not assumptions.
Availability	Pilot SLO 99.5% monthly, tracked with uptime/error monitoring; not a contractual SLA. Evidence-processing failures must not lose submitted records.
Recovery	Proposed RPO ≤ 24 h and RTO ≤ 8 h. Scheduled database and separate object backups; restore drill before production and quarterly thereafter [17].
Browser support	Test current and previous major Chrome, Edge, Firefox and Safari; include Android Chrome and iOS Safari. Support 320 px layouts and 200% zoom.
Security	RLS and storage-policy tests; server authorization on every mutation; origin/CSRF protection appropriate to sessions; CSP; output escaping; rate limits; audit review; secrets kept server-side.
Observability	Track API errors/latency, upload failures, scan queue age, overdue reviews, export failures, backup failures and public-removal latency. Logs exclude certificates, tokens and full registration numbers.

Retention and consent: institution-owned decisions

Proposed defaults: purge abandoned uploads after 24 hours; export files after 24 hours; notify before removing drafts inactive for 90 days. Keep approved records, evidence and audit history for the institution-approved retention period. Do not choose an arbitrary permanent-retention policy.

Publication consent is independent of internal recordkeeping. Withdrawal immediately blocks fresh public reads; target cache removal ≤ 60 seconds. Third-party search caches and copies already downloaded may remain outside system control. Avoid public indexing until the college approves the publication policy.

Operational ownership

Appoint a faculty product owner for taxonomy, review SLAs and disputes; an IT owner for accounts, domains, provider approvals and incidents; and a technical maintainer for releases/backups. Provide short reviewer and student guides plus a handover runbook. At least two authorized administrators should have recovery access.

Delivery plan, budget & decisions

Planning estimate: 8–10 weeks for two developers already familiar with React/TypeScript, with part-time faculty feedback. A solo student learning the backend should plan roughly 12–16 weeks. These are scope-based estimates, not commitments.

Phase	Deliverable and exit gate
1 • Week 1	Confirm fields, policies and report template; wireframes; test login, private upload and one protected API end to end. Choose stack and deployment region.
2 • Weeks 2–3	Accounts, profiles, RLS, drafts, validated form, upload processing and submission. Exit: ownership and retry tests pass.
3 • Weeks 4–5	Reviewer queue, decisions, revisions, audit and consent-based publishing. Exit: workflow and unpublish tests pass.
4 • Weeks 6–7	Public gallery, filters, accessible loading states, CSV/PDF reports and operational dashboards. Exit: report totals reconcile.
5 • Weeks 8–10	Security/accessibility/performance checks, backup restore, two-week pilot and fixes. Launch only after faculty accepts sample reports and privacy behavior.

Budget and capacity assumptions

Set an initial planning allowance of ₹5,000–₹12,000/month for production hosting, managed database/storage, email, scanning/worker capacity and monitoring. This is a contingency budget, not researched vendor pricing. Obtain current quotes before procurement; free-tier availability must not be treated as a production guarantee.

Example storage model: 5,000 records × (2 MB evidence + 1 MB photo derivatives) ≈15 GB before backups, revisions and logs. Reserve 2–3× headroom. Track storage, download bandwidth, email volume and export-worker time monthly. Existing college infrastructure may change both cost and operations burden.

Decisions required before implementation

Faculty/product owner: final name; institution branding; mandatory evidence; who verifies; review SLA; team member consent; public audience; report format; academic-year boundaries; retention. IT owner: college-email/roster access, SSO need, hosting region, budget and recovery owner. Course coordinator: whether Java is mandatory.

Definition of done

All P0 flows pass; no unresolved critical/high security defects; keyboard and screen-reader smoke tests pass; slow-network recovery works; representative reports approved; restore demonstrated; monitoring and runbook ready. Release P1 reactions only after the core workflow is stable.

Research references • 1 of 2

Accessed 6 October 2026. References support documented capabilities; specific product choices, timings, budgets and acceptance thresholds are recommendations unless explicitly attributed. Links are clickable. Exact dependencies must be revalidated at build kickoff.

[1] Next.js — Server and Client Components

Server rendering and the client interactivity boundary.

<https://nextjs.org/docs/app/getting-started/server-and-client-components>

[2] shadcn/ui — Component system

Customizable components and the basis for the proposed design system.

<https://ui.shadcn.com/>

[3] shadcn/ui — React Hook Form

Field composition, React Hook Form and Zod validation patterns.

<https://ui.shadcn.com/docs/forms/react-hook-form>

[4] shadcn/ui — Spinner

Action-level loading indicator; sizing and button usage.

<https://ui.shadcn.com/docs/components/base/spinner>

[5] shadcn/ui — Skeleton

Placeholder component for content loading.

<https://ui.shadcn.com/docs/components/base/skeleton>

[6] Supabase — Row Level Security

Database access policies; not a replacement for application authorization design.

<https://supabase.com/docs/guides/database/postgres/row-level-security>

[7] Supabase — Storage Access Control

Storage authorization through policies.

<https://supabase.com/docs/guides/storage/security/access-control>

[8] MDN — scroll-behavior

Native smooth behavior for navigation/programmatic scrolling.

<https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/scroll-behavior>

[9] Motion — Accessibility

Respecting user reduced-motion preferences.

<https://motion.dev/docs/react-accessibility>

[10] Lenis — Official repository

Optional JavaScript smooth scrolling; evaluated but excluded from MVP.

<https://github.com/darkroomengineering/lenis>

Research references • 2 of 2

[11] TanStack Table — Introduction

Headless table logic; the project supplies markup and styling.

<https://tanstack.com/table/v8/docs/introduction>

[12] Spring Boot — Official reference

Alternative Java backend foundation.

<https://docs.spring.io/spring-boot/index.html>

[13] Material UI — Overview

Alternative React component library based on Material Design.

<https://mui.com/material-ui/getting-started/>

[14] OWASP — File Upload Cheat Sheet

Upload validation, storage separation and file-processing controls.

https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

[15] Playwright — Installation and testing

Browser automation for critical end-to-end workflows.

<https://playwright.dev/docs/intro>

[16] Supabase — Server-side client setup

SSR authentication integration; verify current session and caching guidance.

<https://supabase.com/docs/guides/auth/server-side/nextjs>

[17] Supabase — Database Backups

Backup behavior and the distinction between database records and Storage objects.

<https://supabase.com/docs/guides/platform/backups>

[18] W3C WAI — WCAG overview

Accessibility standards used to define the project target.

<https://www.w3.org/WAI/standards-guidelines/wcag/>

[19] web.dev — Web Vitals

LCP, INP and CLS definitions and good-experience thresholds.

<https://web.dev/articles/vitals>

Requirement traceability to the sketch

Student inputs → Sections 7–8. Participation/rank and four categories → Sections 7 and 9. Public achievement pictures and navigation arrows → Section 9. Reactions → Section 9, P1. Faculty consolidated reports → Section 10. Remove action → Sections 6 and 16. Added verification and privacy requirements → Sections 5–8 and 13–16.

This document specifies the product to build. It does not claim an application has already been implemented, benchmarked or approved by the college.